More Fundamental Programming Concepts

In the following sections, more universal programming concepts will be introduced. These concepts are used in many programming languages, with a few syntactical differences. As I introduce these concepts, I will integrate them into pseudo-code examples using C-like syntax. By the end, the pseudo code should look very similar to C code.

Variables

The counter used in the for loop is actually a type of variable. A *variable* can simply be thought of as an object that holds data that can be changed— hence the name. There are also variables that don't change, which are aptly called *constants*. Returning to the driving example, the speed of the car would be a variable, while the color of the car would be a constant. In pseudo code, variables are simple abstract concepts, but in C (and in many other languages), variables must be declared and given a type before they can be used. This is because a C program will eventually be compiled into an executable program. Like a cooking recipe that lists all the required ingredients before giving the instructions, variable declarations allow you to make preparations before getting into the meat of the program. Ultimately, all variables are stored in memory somewhere, and their declarations allow the compiler to organize this memory more efficiently. In the end though, despite all of the variable type declarations, everything is all just memory.

In C, each variable is given a type that describes the information that is meant to be stored in that variable. Some of the most common types are `int` (integer values), `float` (decimal floating-point values), and `char` (single character values). Variables are declared simply by using these keywords before listing the variables, as you can see below.

```
int a, b;
float k;
char z;
```

The variables a and b are now defined as integers, k can accept floating point values (such as 3.14), and z is expected to hold a character value, like *A* or *w*. Variables can be assigned values when they are declared or anytime afterward, using the = operator.

```
int a = 13, b;
float k;
char z = 'A';

k = 3.14;
z = 'w';
b = a + 5;
```

After the following instructions are executed, the variable a will contain the value of 13, k will contain the number 3.14, z will contain the character *w*, and b will contain the value 18, since 13 plus 5 equals 18. Variables are simply a way to remember values; however, with C, you must first declare each variable's type.

Arithmetic Operators

The statement `b = a + 7` is an example of a very simple arithmetic operator. In C, the following symbols are used for various arithmetic operations.

The first four operations should look familiar. Modulo reduction may seem like a new concept, but it's really just taking the remainder after division. If a is 13, then 13 divided by 5 equals 2, with a remainder of 3, which means that `a % 5 = 3`. Also, since the variables a and b are integers, the statement `b = a / 5` will result in the value of 2 being stored in b, since that's the integer portion of it. Floating-point variables must be used to retain the more correct answer of 2.6.

Operation	Symbol	Example

Addition	+	`b = a + 5`
Subtraction	-	`b = a - 5`
Multiplication	*	`b = a * 5`
Division	/	`b = a / 5`
Modulo reduction	%	`b = a % 5`

To get a program to use these concepts, you must speak its language. The C language also provides several forms of shorthand for these arithmetic operations. One of these was mentioned earlier and is used commonly in for loops.

Full Expression	Shorthand	Explanation
`i = i + 1`	`i++ or ++i`	Add 1 to the variable.
`i = i - 1`	`i-- or --i`	Subtract 1 from the variable.

These shorthand expressions can be combined with other arithmetic operations to produce more complex expressions. This is where the difference between `i++` and `++i` becomes apparent. The first expression means *Increment the value of `i` by 1* after *evaluating the arithmetic operation*, while the second expression means *Increment the value of `i` by 1* before *evaluating the arithmetic operation*. The following example will help clarify.

```
int a, b;
a = 5;
b = a++ * 6;
```

At the end of this set of instructions, `b` will contain 30 and `a` will contain 6, since the shorthand of `b = a++ * 6;` is equivalent to the following statements:

```
b = a * 6;
a = a + 1;
```

However, if the instruction `b = ++a * 6;` is used, the order of

the addition to a changes, resulting in the following equivalent instructions:

```
a = a + 1;
b = a * 6;
```

Since the order has changed, in this case b will contain 36, and a will still contain 6.

Quite often in programs, variables need to be modified in place. For example, you might need to add an arbitrary value like 12 to a variable, and store the result right back in that variable (for example, `i = i + 12`). This happens commonly enough that shorthand also exists for it.

Full Expression	Shorthand	Explanation
`i = i + 12`	`i+=12`	Add some value to the variable.
`i = i - 12`	`i-=12`	Subtract some value from the variable.
`i = i * 12`	`i*=12`	Multiply some value by the variable.
`i = i / 12`	`i/=12`	Divide some value from the variable.

Comparison Operators

Variables are frequently used in the conditional statements of the previously explained control structures. These conditional statements are based on some sort of comparison. In C, these comparison operators use a shorthand syntax that is fairly common across many programming languages.

Condition	Symbol	Example
Less than	<	`(a < b)`
Greater than	>	`(a > b)`
Less than or equal to	<=	`(a <= b)`

Greater than or equal to	>=	`(a >= b)`
Equal to	==	`(a == b)`
Not equal to	!=	`(a != b)`

Most of these operators are self-explanatory; however, notice that the shorthand for *equal to* uses double equal signs. This is an important distinction, since the double equal sign is used to test equivalence, while the single equal sign is used to assign a value to a variable. The statement `a = 7` means *Put the value 7 in the variable a*, while `a == 7` means *Check to see whether the variable a is equal to 7*. (Some programming languages like Pascal actually use := for variable assignment to eliminate visual confusion.) Also, notice that an exclamation point generally means *not*. This symbol can be used by itself to invert any expression.

```
!(a < b)    is equivalent to    (a >= b)
```

These comparison operators can also be chained together using shorthand for OR and AND.

Logic	Symbol	Example
OR	||	`((a < b) || (a < c))`
AND	&&	`((a < b) && !(a < c))`

The example statement consisting of the two smaller conditions joined with OR logic will fire true if `a` is less than `b`, OR if `a` is less than `c`. Similarly, the example statement consisting of two smaller comparisons joined with AND logic will fire true if `a` is less than `b` AND `a` is not less than `c`. These statements should be grouped with parentheses and can contain many different variations.

Many things can be boiled down to variables, comparison operators, and control structures. Returning to the example of the mouse searching for food, hunger can be translated into a

Boolean true/false variable. Naturally, 1 means true and 0 means false.

```
While (hungry == 1)
{
  Find some food;
  Eat the food;
}
```

Here's another shorthand used by programmers and hackers quite often. C doesn't really have any Boolean operators, so any nonzero value is considered true, and a statement is considered false if it contains 0. In fact, the comparison operators will actually return a value of 1 if the comparison is true and a value of 0 if it is false. Checking to see whether the variable `hungry` is equal to 1 will return 1 if `hungry` equals 1 and 0 if `hungry` equals 0. Since the program only uses these two cases, the comparison operator can be dropped altogether.

```
While (hungry)
{
  Find some food;
  Eat the food;
}
```

A smarter mouse program with more inputs demonstrates how comparison operators can be combined with variables.

```
While ((hungry) && !(cat_present))
{
  Find some food;
  If(!(food_is_on_a_mousetrap))
    Eat the food;
}
```

This example assumes there are also variables that describe the presence of a cat and the location of the food, with a value of 1 for true and 0 for false. Just remember that any nonzero value is considered true, and the value of 0 is considered false.

Functions

Sometimes there will be a set of instructions the programmer knows he will need several times. These instructions can be

grouped into a smaller subprogram called a *function*. In other languages, functions are known as subroutines or procedures. For example, the action of turning a car actually consists of many smaller instructions: Turn on the appropriate blinker, slow down, check for oncoming traffic, turn the steering wheel in the appropriate direction, and so on. The driving directions from the beginning of this chapter require quite a few turns; however, listing every little instruction for every turn would be tedious (and less readable). You can pass variables as arguments to a function in order to modify the way the function operates. In this case, the function is passed the direction of the turn.

```
Function Turn(variable_direction)
{
  Activate the variable_direction blinker;
  Slow down;
  Check for oncoming traffic;
  while(there is oncoming traffic)
  {
    Stop;
    Watch for oncoming traffic;
  }
  Turn the steering wheel to the variable_direction;
  while(turn is not complete)
  {
    if(speed < 5 mph)
      Accelerate;
  }
  Turn the steering wheel back to the original position;
  Turn off the variable_direction blinker;
}
```

This function describes all the instructions needed to make a turn. When a program that knows about this function needs to turn, it can just call this function. When the function is called, the instructions found within it are executed with the arguments passed to it; afterward, execution returns to where it was in the program, after the function call. Either left or right can be passed into this function, which causes the function to turn in that direction.

By default in C, functions can return a value to a caller. For

those familiar with functions in mathematics, this makes perfect sense. Imagine a function that calculates the factorial of a number—naturally, it returns the result.

In C, functions aren't labeled with a "function" keyword; instead, they are declared by the data type of the variable they are returning. This format looks very similar to variable declaration. If a function is meant to return an integer (perhaps a function that calculates the factorial of some number *x*), the function could look like this:

```
int factorial(int x)
{
  int i;
  for(i=1; i < x; i++)
    x *= i;
  return x;
}
```

This function is declared as an integer because it multiplies every value from 1 to *x* and returns the result, which is an integer. The return statement at the end of the function passes back the contents of the variable *x* and ends the function. This factorial function can then be used like an integer variable in the main part of any program that knows about it.

```
int a=5, b;
b = factorial(a);
```

At the end of this short program, the variable b will contain 120, since the factorial function will be called with the argument of 5 and will return 120.

Also in C, the compiler must "know" about functions before it can use them. This can be done by simply writing the entire function before using it later in the program or by using function prototypes. A *function prototype* is simply a way to tell the compiler to expect a function with this name, this return data type, and these data types as its functional arguments. The actual function can be located near the end of the program, but it can be used anywhere else, since the compiler already knows about it. An example of a function prototype for the

`factorial()` function would look something like this:

```
int factorial(int);
```

Usually, function prototypes are located near the beginning of a program. There's no need to actually define any variable names in the prototype, since this is done in the actual function. The only thing the compiler cares about is the function's name, its return data type, and the data types of its functional arguments.

If a function doesn't have any value to return, it should be declared as `void`, as is the case with the `turn()` function I used as an example earlier. However, the `turn()` function doesn't yet capture all the functionality that our driving directions need. Every turn in the directions has both a direction and a street name. This means that a turning function should have two variables: the direction to turn and the street to turn on to. This complicates the function of turning, since the proper street must be located before the turn can be made. A more complete turning function using proper C-like syntax is listed below in pseudo-code.

```
void turn(variable_direction, target_street_name)
{
  Look for a street sign;
  current_intersection_name = read street sign name;
  while(current_intersection_name != target_street_name)
  {
    Look for another street sign;
    current_intersection_name = read street sign name;
  }

  Activate the variable_direction blinker;
  Slow down;
  Check for oncoming traffic;
  while(there is oncoming traffic)
  {
    Stop;
    Watch for oncoming traffic;
  }
  Turn the steering wheel to the variable_direction;
  while(turn is not complete)
  {
    if(speed < 5 mph)
```

```
        Accelerate;
    }
    Turn the steering wheel right back to the original position;
    Turn off the variable_direction blinker;
  }
```

This function includes a section that searches for the proper intersection by looking for street signs, reading the name on each street sign, and storing that name in a variable called `current_intersection_name`. It will continue to look for and read street signs until the target street is found; at that point, the remaining turning instructions will be executed. The pseudo-code driving instructions can now be changed to use this turning function.

```
Begin going East on Main Street;
while (there is not a church on the right)
   Drive down Main Street;
if (street is blocked)
{
  Turn(right, 15th Street);
  Turn(left, Pine Street);
  Turn(right, 16th Street);
}
else
  Turn(right, 16th Street);
Turn(left, Destination Road);
for (i=0; i<5; i++)
  Drive straight for 1 mile;
Stop at 743 Destination Road;
```

Functions aren't commonly used in pseudo-code, since pseudo-code is mostly used as a way for programmers to sketch out program concepts before writing compilable code. Since pseudo-code doesn't actually have to work, full functions don't need to be written out—simply jotting down *Do some complex stuff here* will suffice. But in a programming language like C, functions are used heavily. Most of the real usefulness of C comes from collections of existing functions called *libraries*.